

Observabilidad en el Toolchain de Desarrollo

Arquitectura del Software

Carolina González García - Jorge Suárez Álvarez

Índice de contenidos

01

¿Qué es la observabilidad en el toolchain?

06

Justificación de negocio (ROI)

02

Los tres pilares: Logs, Métricas y Trazas

07

Cómo empezar: greenfield y sistemas legacy

03

Ciclos de feedback del desarrollador

08

Retos organizativos y culturales

04

Tests flaky y gestión de fallos

09

El futuro: seguridad de la cadena de suministro

05

IA en el toolchain: retos y oportunidades

¿Qué es la observabilidad en el toolchain?

Definición

Aplicar a nuestras herramientas de construcción de software (compiladores, CI/CD, gestores de dependencias, tests...) las mismas técnicas de monitorización que usamos en aplicaciones de producción.

"Tratar el toolchain como sistema de producción"

App en producción	Toolchain
Logs de app	Logs de build
Métricas de latencia	Tiempo de compilación
Alertas de caída	Alertas de build rota
Trazas de requests	Trazas de pipeline

Los tres pilares de la observabilidad aplicados al build

LOGS

Cada herramienta genera logs. El reto es filtrar la información relevante de millones de líneas de compilación, linting y tests.

Ejemplos:

- Warnings del compilador
- Errores de dependencias
- Salida de herramientas de seguridad

MÉTRICAS

Datos cuantificables que permiten medir el estado del sistema a lo largo del tiempo y detectar regresiones.

Ejemplos:

- Tiempo total de build
- % de tests flaky
- Nº de warnings acumulados

TRAZAS

Permiten identificar cuellos de botella dentro del pipeline, desglosando qué fase consume más tiempo.

Ejemplos:

- Tiempo de compilación vs. test
- Descarga de dependencias
- Análisis estático (linting)

Ciclos de feedback del desarrollador

20 min

*build medio en
mucho equipos*

10 min

*coste cognitivo de
cada cambio de contexto*

El bucle del desarrollador bloqueado

1. Lanzas el build (20 min)
2. Sabes que tardará → revisas el email
3. Tu cerebro tarda 10 min en enfocarse en el email
4. El build falla a los 6 min
5. Tardas 10 min en volver a enfocarte en el código
6. Corriges → repites el ciclo

Sin datos sobre el tiempo de build → no puedes saber ni medir ni mejorar

Tests flaky: el ruido que paraliza equipos

Test flaky (o test inestable): un test que pasa en un momento y falla en otro sin cambios en el código — o que funciona localmente pero no en CI.

El problema

- Desarrolladores pierden horas investigando fallos que no son suyos
- Desconfianza en el sistema de CI
- Ruido que oculta errores reales

Detección con observabilidad

- Registrar qué tests fallan y con qué frecuencia
- Alertar cuando el % supera un umbral (ej. >2%)
- Correlacionar fallos con cambios de infraestructura

Acciones posibles

- Poner el test en cuarentena automáticamente
- Abrir una issue y asignarla al equipo responsable
- Reintentar el test como medida temporal

IA en el toolchain: nuevos retos de observabilidad



Volumen de código

La IA genera código masivamente → builds más largas, más warnings, más tests necesarios.



Procedencia (Provenance)

¿De dónde vino este código? ¿Fue generado por IA o por un humano? Crucial para seguridad y auditoría.



Tests generados sin valor

Tests autogenerados del mismo origen que el código no validan nada real. Se necesita supervisión humana.



¿Mejora la productividad?

"Los desarrolladores dicen que les gusta" no es un dato. Hay que medir: PRs, tiempo a producción, features entregadas.



Supply chain security

Más código entrando al sistema = mayor superficie de ataque. Herramientas SLSA e in-toto para verificar origen.



IA para la propia observabilidad

Clasificar automáticamente tipos de fallo: infraestructura vs. código vs. tests flaky. Automatizar reintentos.

Justificación de negocio: ¿por qué invertir?

Caso real (Gradle)

Situación: Los builds de Gradle corrían más rápido en Apple Silicon que en Intel.

Acción: Compraron portátiles nuevos para todos los desarrolladores.

Resultado: La empresa ahorró dinero, los devs recibieron hardware nuevo. Win-win.

Más features / sprint

↑ **Productividad**

Resolución más rápida de problemas en el build

↓ **MTTR**

Builds sobredimensionadas detectadas

↓ **Costes Cloud**

Desarrollador en "estado de flujo" más tiempo

↑ **Satisfacción**

Métricas clave para empezar

Brian Demers recomienda empezar simple. La primera métrica más importante:



Tiempo de build

Fácil de medir, fácil de comunicar a negocio, indica dónde centrar esfuerzos.

% de tests flaky

Alertar si supera el 2%

Nº de warnings del compilador

Debe reducirse con el tiempo, no crecer

Tiempo hasta primer fallo (TTFF)

Idealmente en los primeros segundos

Tiempo de descarga de dependencias

Indica falta de caché configurado

Frecuencia de builds locales

Indicador de cuánto trabajan los desarrolladores

Uso de CPU/memoria en agentes CI

Detectar sobre/infra-provisionamiento

Cómo empezar: proyectos greenfield

1

Instrumentar todo desde el inicio

Activar toda la recopilación de datos disponible en cada herramienta. Es el momento más barato de hacerlo.

2

Montar la infraestructura de colección

Aunque no uses los datos de inmediato, tenerlos recogidos es una inversión de conocimiento para el equipo.

3

Definir umbrales y alertas accionables

No sirve de nada recoger datos que nadie mira. Si el % de tests flaky supera el 2%, que alguien lo sepa.

4

Asignar responsabilidad

Crear issues automáticas, asignarlas a alguien. La observabilidad sin acción es ruido.

5

Escalar a otros proyectos

Una vez el equipo aprende, replica el sistema a otros repositorios/proyectos. El conocimiento organizacional escala.

Cómo empezar: sistemas legacy

La ventaja del legacy es que ya tienes problemas visibles. Empieza por los que más duelen.

¿Qué medir primero?

- ¿Qué % del build son tests?
- ¿Cuánto tarda el linting (se puede mover al IDE)?
- ¿Hay deuda técnica del build (ANT sin migrar)?
- ¿Versiones mezcladas de dependencias?

Estrategia incremental

- No busques solucionar todo a la vez
- Si tienes 1000 issues de seguridad, que no suban de 1000
- Cada mejora medida con datos, antes y después
- Simplificar el toolchain también es una opción

"¿Es esto lento?" → Sí. Pero ¿por qué y dónde? Para eso necesitas datos.

Retos organizativos y culturales

"No es mi problema"

En equipos grandes, los builds rotos se ignoran porque nadie se siente responsable de la infraestructura compartida.

Asignar responsabilidad clara del toolchain y crear alertas directas al equipo responsable.

Falta de conocimiento

A diferencia de la observabilidad en producción, la observabilidad del toolchain no tiene playbooks establecidos en la mayoría de empresas.

Transferir el conocimiento de DevOps/SRE al toolchain: tratar el build como sistema productivo.

Dificultad de acceso a infraestructura

Pedir nuevas métricas o cambios en CI requiere pasar por equipos centrales que no comprenden el problema.

Demostrar el ROI con datos simples primero: tiempos de build antes/después.

Datos sin acción

Coleccionar datos no sirve de nada si nadie los mira o actúa sobre ellos.

Ejemplo real: para abrir un ticket de CI, obligar a adjuntar el Build Scan → el dato existe y fuerza el autodiagnóstico.

El futuro: seguridad de la supply chain de software

La observabilidad del toolchain converge con la seguridad de la cadena de suministro de software (Software Supply Chain Security).

SLSA

Supply-chain Levels for Software Artifacts

Marco de Google para verificar la procedencia y seguridad del proceso de build, desde el código fuente hasta el binario.

in-toto

Framework de atestiguamiento

Permite definir y verificar políticas sobre qué pasos del pipeline se ejecutaron, en qué máquinas y con qué entradas/salidas.

Los mismos datos que necesitas para observabilidad del toolchain (qué entra, qué sale, en qué máquina se ejecutó) son los que necesitas para la seguridad de la cadena de suministro.

Casos de uso reales mencionados en el podcast

Agentes CI sobredimensionados

-  **Hallazgo:** Solo usaban el 10-25% de CPU
-  **Acción:** Cambiar a instancias más pequeñas en Cloud
-  **Resultado:** Ahorro significativo de costes

Build más lenta tras migrar de Jenkins a GitHub Actions

-  **Hallazgo:** Los builds efímeros no tenían el caché de dependencias configurado
-  **Acción:** Configurar el caché de dependencias
-  **Resultado:** Build volvió a su velocidad original

10.000+ repositorios a gestionar

-  **Hallazgo:** Imposible coordinar actualizaciones de versiones de Java
-  **Acción:** Herramienta interna de inventario + observabilidad
-  **Resultado:** Visibilidad de a qué repositorios afecta cada actualización

¿Intel Mac vs Apple Silicon?

-  **Hallazgo:** Builds más rápidos en Apple Silicon (medición interna)
-  **Acción:** Compra de portátiles nuevos para los desarrolladores
-  **Resultado:** Mayor productividad + desarrolladores contentos

Conclusiones

- 1 El toolchain es tan complejo como una app de producción — pero casi nadie lo monitoriza.
- 2 Los ciclos de feedback cortos son clave para la productividad: cada minuto de espera tiene un coste cognitivo.
- 3 Los tres pilares (logs, métricas, trazas) se aplican igual al build que a los sistemas en producción.
- 4 La IA aumenta el volumen de código y la necesidad de observabilidad, no la reduce.
- 5 El ROI es real y medible: menos tiempo perdido, costes reducidos, desarrolladores más satisfechos.
- 6 Empieza pequeño: mide el tiempo de build, pon alertas, asigna responsabilidad.
- 7 El futuro une observabilidad con la seguridad de la cadena de suministro (SLSA, in-toto).

**"Empieza a tratar tu toolchain
como un sistema en producción."**

— *Brian Demers, Software Engineering Radio*

Fuente: Software Engineering Radio — Episodio con Brian Demers

<https://www.se-radio.net>